

用 AI 創作遊戲



戰國麻將列傳

6MB

單一 HTML 檔

30+

次對話回合

0行

手寫程式碼

完整分享，適合對 AI 開發遊戲有興趣的人

填寫問卷



Allen Kao 高偉倫

DILA Digital Archives Group

學歷：

國立台灣大學電機工程學研究所

經歷：

Acer Inc. R&D

Nvidia GFX SW FAE

Intel AE/TME

Intel AE Manager



緣起

36年前的願望

傳統開發遊戲的方法



今天要聊什麼

01

AI 開發是什麼？

和傳統寫程式有何不同

5分

02

從零到遊戲完成：8個里程碑

一步步的開發歷程

15分

03

實際碰到的 Bug

AI 犯了哪些錯？怎麼修？

10分

04

AI 幫了什麼？

真正節省了多少時間與精力

8分

05

你也可以這樣做

入門建議與注意事項

5分



第一章

什麼是 AI 輔助開發？

😓 傳統開發方式

- 要先學會語言語法
- 查文件、Stack Overflow
- 逐行手寫程式碼
- Debug 靠自己猜
- 要知道「怎麼做」



🤖 AI 輔助開發

- 用自然語言描述需求
- AI 理解後直接產生程式碼
- 你審查、測試、調整方向
- AI 找出錯誤並修復
- 只需要知道「要什麼」



✅ 這個遊戲的開發：我完全沒有手動寫過任何一行程式碼 — 全部由 Claude AI 生成

《戰國麻將列傳》介紹

遊戲背景

春秋戰國七雄以麻將決勝負，征服六國、統一天下。玩家選擇一國，逐一挑戰其餘六國。



13張麻將

吃碰槓完整規則



10種特殊牌型

九連寶燈、國士無雙



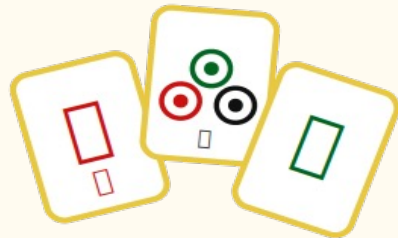
存檔繼續

localStorage 記錄進度



AI生成背景

各國專屬戰場插圖



開發歷程總覽 · 8 個里程碑



1

遊戲雛形

概念 → 完整HTML遊戲

2

基礎功能

Canvas牌面 · 吃碰槓

3

特殊牌型

10種牌型 + 背景插圖

4

視覺升級

AI插圖嵌入各國背景

5

勝負流程

勝利動畫 + 存檔系統

6

Bug修復

多輪JS結構 + 邏輯修正

7

牌面標準化

用戶上傳圖片替換牌面

8

體驗優化

自動理牌 + 偏向發牌

里程碑 1-2

從一句話到完整遊戲



第一句 Prompt (原文)

"幫我寫一個遊戲，戰國麻將列傳。春秋戰國時代為背景，各小國利用麻將對奕決勝負，相互爭戰最後統一天下。玩家可以選擇任一個小國開始遊戲，使用十三張麻將。"



AI 一次生成了什麼？



完整遊戲架構

七國選擇、對戰流程



麻將邏輯

13張、胡牌、吃碰槓



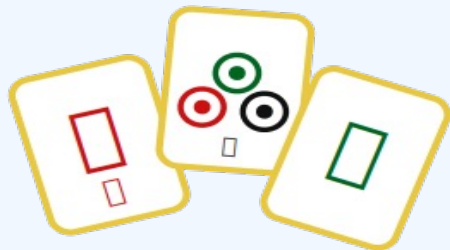
Canvas 繪製

牌面、動畫、UI



響應式佈局

手機電腦皆可玩



第二輪：「可以畫出麻將牌嗎？另外沒辦法吃碰槓」→ AI 立即補上 Canvas 繪牌 + 完整系統



齊國 | 攻佔：0 國 | 對手：楚國

我方回合

第1局 / 楚國

諸侯形勢

楚 楚國 ✕

燕 燕國

韓 韓國

魏 魏國

趙 趙國

秦 秦國

特殊牌型

✦ 九連寶燈 88番

✦ 國士無雙 88番

✦ 清一色 24番

✦ 字一色 64番

✦ 對對胡 22番

✦ 七對子 24番

✦ 大四喜 88番

✦ 綠一色 88番

✦ 混一色 8番

✦ 小三元 32番

敵方手牌

副露

一萬

二萬

三萬

四萬

五萬

六萬

七萬

八萬

九萬

棄牌

一萬

南

西

西

四萬

八萬

二萬

三萬

四萬

五萬

六萬

七萬

八萬

九萬

牌山：88 張

摸牌

我方手牌（點選後棄牌）

一萬

二萬

三萬

四萬

五萬

六萬

七萬

八萬

九萬

副露

一萬

二萬

三萬

四萬

五萬

六萬

七萬

八萬

九萬

棄牌

南

九萬

三萬

中

西

六萬

五萬

四萬

三萬

二萬

一萬

認輸

敵棄：4 張

摸牌：3 條

✦ 可胡牌！



特殊牌型 + 視覺升級

九連寶燈

88
分

國士無雙

88
分

大四喜

88
分

綠一色

88
分

字一色

64
分

清一色

24
分

七對子

24
分

對對胡

22
分

小三元

32
分

混一色

8分

🌟 視覺升級

主畫面

AI戰國插圖背景

選國畫面

七國君主站排圖

各國對戰

秦趙楚等專屬場景

水墨插圖

Canvas自動生成

粒子特效

勝利時金色煙火

3D牌面

立體浮雕麻將牌

里程碑 5

勝負流程 + 存檔系統



🎮 完整遊戲流程

選擇國家

分析手牌

選注倍率

對戰麻將

勝利閃光

繼續征戰

統一天下！

💾 存檔系統設計

存檔時機

每次勝利後自動儲存，敗北也儲存當前進度

讀取方式

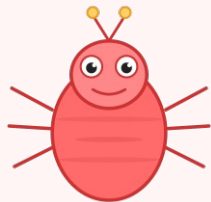
下次開啟自動偵測，彈出「發現存檔」視窗

存放位置

瀏覽器 localStorage，單一 key 存所有狀態

完成後

統一中國後自動刪除存檔（任務完成！）



AI 也會犯錯 —— 真實 Bug 案例

BUG 1



嚴重

JS 結構崩潰

原因：存檔系統插入時，程式碼被包進 `showSel()` 函數內部

修法：找出多餘的 `}` 括號，將 `save/load` 移到正確的全域位置

BUG 2



中等

胡牌排序錯誤

原因：`canFNS()` 用 `Object.keys()` 的插入順序，非數字順序

修法：所有牌型組合函數加入按數字排序：`keys.sort()`

BUG 3



中等

偏向發牌結構無效

原因：字一色偏向只隨機取13張字牌，未保證正確牌型結構

修法：每種特殊牌型改為先建構「1對+4刻」的有效結構再發牌

用戶上傳圖片替換牌面



🔧 技術重點

問題：Canvas `drawImage()` 在圖片未載入時呼叫 → 牌面空白

修法：頁面載入時預先建立所有 `Image` 物件，全部完成後觸發 `renderAll()`

規模：27 張牌面圖（9萬+9筒+9條），全部 Base64 嵌入 HTML

第四章

AI 真正幫了什麼？

~40小時

估計傳統開發時間

~3小時

實際 AI 輔助時間

93%

時間節省



AI 擅長的事

- 快速生成大量程式碼
- 記住複雜的語法細節
- 找出並修復 Bug
- 解釋程式碼邏輯
- 重構和優化程式

你需要做的事

- 描述清楚你要什麼
- 測試和驗證結果
- 判斷方向是否正確
- 提供用戶回饋
- 做最終的決定



AI 的限制與正確使用方式



AI 會犯錯，而且不知道它犯了錯

括號位置錯誤、插入點不對、邏輯 bug —— 每次都需要人工驗證結果。



對話越長越容易混亂

超過 30 輪對話，AI 有時會忘記之前的設計，需要定期提醒或重新說明。



你的描述品質決定輸出品質

「幫我修好程式」vs「摸牌後自動選取剛摸到的牌為預設棄牌」—— 越具體越準確。



AI 是工具，不是替代品

你仍需要理解「要做什麼」—— AI 負責「怎麼做」。沒有方向感，AI 也幫不上忙。

給同學的 5 個建議

01

先做再學

不需要先學完語法再開始。有想法就直接跟 AI 說，在做的過程中學習。

02

描述要具體

把需求說得像在跟工程師解釋：「我要 A，當 B 發生時，需要做 C」。

03

每次都要測試

AI 給你的程式，自己要跑一遍。發現錯誤直接截圖或描述給 AI，它會修。

04

不要怕改需求

做到一半想改方向很正常。AI 不會生氣，直接告訴它你想要什麼不同。

05

記錄你的過程

把每個重要改動截圖存起來。在開發過程中若遇到邏輯前後衝突時，幫助自己分析。





你可以用 AI 做什麼？

入門

- 個人作品集網頁
- 簡單的待辦事項 App
- 計算機、換算工具
- 簡單的猜數字遊戲

進階

- 桌遊 / 卡牌遊戲
- 資料視覺化圖表
- 學校課表管理系統
- 簡易記帳 App

挑戰

- 多人連線小遊戲
- AI 聊天機器人介面
- 影片剪輯工具 UI
- 完整的網頁應用



重點不是技術門檻，而是你有沒有想做某件事的動力。AI 幫你解決技術，你負責想像力！

結語



AI 不會取代你



它會讓有想法的你，更加強大

💡 一個有趣的想法 + AI 的執行力 = 真實的成品

🌱 你不需要先成為工程師，才能開始創造

🎯 最重要的技能：清楚表達你要什麼

謝謝大家 · 歡迎提問 🙌

它會讓有想法的你，更加強大

